7장
문자 기준으로 데이터 묶기: 딕셔너리

앞 장에서는 리스트를 써서 데이터를 정리하고, 정렬하는 방법을 배웠습니다. 리스트는 매우 유용한 데이터 타입입니다. 데이터를 일정 기준에 따라 정리하고, 인덱스를 활용해 만들 것 수록을 쓸 수 있기 때문이죠.

그런데 리스트를 사용할 때 다소 불편한 점이 있습니다. 리스트의 요소에 접근하려면 인덱스를 사용해야 합니다. 즉 인덱스가 몇 개 없으면 다행이지만, 실 데이터에는 훨씬 많은 개수로 이루어져 있기도 합니다. 이럴 때는 인덱스로 접근하기가 어렵습니다.

이런 리스트의 단점을 보완하는 딕셔너리라는 데이터 타입이 있습니다. 이 장에서는 딕셔너리에 대해 살펴봅니다.

7.1

사전처럼 단어와 의미 연결하기

7.1.1 딕셔너리란

딕셔너리(dictionary)는 우리말로 '사전'이라는 뜻입니다. 파이썬의 딕셔너리는 사전과 비슷한 방식의 데이터 타입입니다. 영한사전을 한번 생각해 볼까요? 영단어가 먼저 나오고 우리말 뜻이 나옵니다. br로 시작하는 단어를 검색하면 break, bring, brush, brave와 같은 영단어가 나오고, 그다음 각 단어의 뜻이 나옵니다.

구조를 이해하기 쉽게 각 단어와 의미를 연결해 표로 만들어 보겠습니다.

▼ 표 7-1 br로 시작하는 단어와 뜻

단어	break	bring	bursh	brave
뜻	깨어지다	가져오다	붓	용감한

영단어를 위에 적고, 우리말 뜻을 아랫줄에 정리했습니다. break는 바로 아래의 '깨어지다'에 대응되고, bring 역시 아래에 있는 '가져오다'와 대응됩니다. 이것이 바로 딕셔너리의 구조입니다. 리스트에서는 값의 위치를 표기하기 위해 숫자만 인덱스로 사용했죠. 하지만 딕셔너리에서는 값의 위치를 나타낼 때 숫자뿐만 아니라 문자도 사용할 수 있습니다. 이 점이 딕셔너리의 가장 큰 특징입니다.

구조를 더 살펴봅시다. 리스트의 인덱스와 같은 역할을 하는 것이 앞의 표에서 break, bring과 같은 영단어입니다. 그런데 이 문자들은 숫자처럼 순서를 나타내지 않기 때문에 인덱스라고 하지 않습니다. 그래서 딕셔너리에서는 break, bring, brush, brave와 같은 문자를 키(key)라고 합니다. 키는 인덱스처럼 데이터의 위치를 알려 주고, 각 데이터에 접근할 수 있는 기준이 됩니다. 그리고 각 키에 해당하는 데이터를 값(value)이라고 합니다.

▼ 표 7-2 딕셔너리의 키와 값

키(key)	break	bring	bursh	brave
값(value)	깨어지다	가져오다	붓	용감한

여기에서 유추할 수 있듯이 딕셔너리에서는 키와 값이 한 쌍입니다. 'break-깨어지다', 'bring-가져오다', 'brush-붓', 'brave-용감한'이라는 네 쌍의 키-값이 있는 것이죠. 딕셔너리는 이처럼 요소를 키와 값 한 쌍으로 표현합니다.

그럼 딕셔너리는 어떻게 만들까요? 딕셔너리는 리스트와 형식이 비슷합니다. 딕셔너리명으로 변수를 만들고 딕셔너리를 저장하면 됩니다. 다만, 리스트가 데이터를 대괄호(`[]`)로 표현했다면 딕셔너리는 중괄호(`{}`)로 표현합니다. 그리고 키-값이 한 쌍이므로 데이터도 한 쌍으로 넣어야 합니다. 한 쌍을 표현하는 방법은 `키 : 값`입니다.

형식

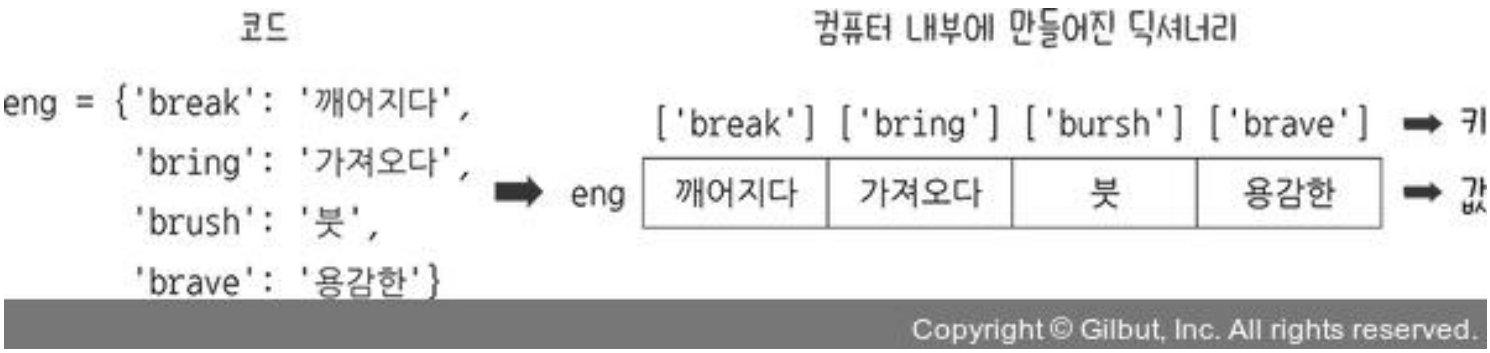
```
딕셔너리명 = {키 : 값 , 키 : 값…}
```

그럼 앞에 나온 네 쌍의 키-값을 저장한 딕셔너리를 만들어 봅시다. 딕셔너리명은 `eng`로 짓겠습니다.

```
eng = {'break': '깨어지다', 'bring': '가져오다', 'brush': '붓', 'brave': '용감한'}
print(eng)
```

```
실행결과
{'break': '깨어지다', 'bring': '가져오다', 'brush': '붓', 'brave': '용감한'}
```

`eng` 딕셔너리를 출력하니 서로 대응되는 키와 값이 콜론(`:`)으로 연결되어 있네요. 화면으로는 이렇게 한 줄로 출력되지만, 실제 컴퓨터 내부에서는 표와 같은 형태로 존재한다고 이해하면 됩니다.



7.1.2 딕셔너리의 키와 값

그럼 이번에는 딕셔너리에 저장된 데이터에 접근해 봅시다. 리스트에서는 인덱스를 활용해 `food[2]`, `spending[-1]` 형태로 리스트의 데이터를 알 수 있었습니다. 딕셔너리도 리스트와 매우 비슷합니다. `리스트명[인덱스]` 형식으로 데이터에 접근했듯이 `딕셔너리명['키']`로 데이터, 즉 값에 접근할 수 있습니다. 예를 들어 앞에 나온 키 `break`의 값을 알고 싶다면 다음과 같이 작성합니다.

```
eng = {'break': '깨어지다', 'bring': '가져오다', 'brush': '붓', 'brave': '용감한'}
print(eng['break'])
```

```
실행결과
깨어지다
```

딕셔너리명 뒤에 키를 붙이니 대응되는 값인 '깨어지다'가 출력됩니다. 리스트에서는 인덱스를 표시하는 대괄호 안에 숫자만 들어갈 수 있었습니다. 하지만 딕셔너리는 키를 기준으로 데이터를 구성하기 때문에 문자도 들어갑니다(키에 문자를 사용하는 경우가 훨씬 많지만 숫자도 사용할 수 있습니다). 그리고 두 번째 줄의 `print(eng['break'])`를 실행하면 딕셔너리에서 키 `break`에 해당하는 값 '깨어지다'를 찾아 출력합니다.

다른 값도 출력해 봅시다.

```
eng = {'break': '깨어지다', 'bring': '가져오다', 'brush': '붓', 'brave': '용감한'}
print(eng['brush'])
print(eng['brave'])
```

```
실행결과
붓
용감한
```

키 `brush`, `brave`와 쌍을 이루는 값이 출력됩니다. 리스트와 공통점, 차이점을 비교하니 딕셔너리도 이해하기 어려운 개념은 아니죠? 인덱스 대신 키를 기준으로 데이터에 접근할 수 있다는 부분이 가장 큰 차이고 그 외에는 비슷한 부분이 많습니다.

그럼 딕셔너리에 존재하지 않는 키를 넣으면 어떻게 될까요? 코드로 살펴봅시다.

```
eng = {'break': '깨어지다', 'bring': '가져오다', 'brush': '붓', 'brave': '용감한'}
print(eng['brief'])
```

```
실행결과
----> 2 print(eng['brief'])
KeyError: 'brief'
```

`KeyError`가 발생합니다. 딕셔너리 개념을 이해했다면 당연히 오류가 난 이유를 알겠죠? `eng` 딕셔너리에 저장된 값에는 `brief`라는 키가 존재하지 않기 때문입니다. 그래서 사용자가 어떤 데이터를 원하는지 컴퓨터가 알 수 없어서 오류가 발생합니다.

또한, 딕셔너리에서는 같은 키가 여러 개 존재할 수 없습니다. 예를 들어, `brush`가 의미하는 '붓'과 '빗'을 둘 다 표현하려고 다음처럼 딕셔너리를 만든다고 합시다.

```
eng = {'break': '깨어지다', 'bring': '가져오다', 'brush': '붓', 'brush': '빗', 'brave': '용감한'}
print(eng['brush'])
```

```
실행결과
빗
```

분명 `brush` 키의 값으로 붓도 저장했는데 왜 빗만 출력될까요? 딕셔너리는 키를 중복 저장하지 않도록 설계됐기 때문입니다. 그래서 키가 같으면 가장 마지막 값만 저장합니다. 따라서 앞에서처럼 같은 키로 다른 값을 넣으면 자동으로 `brush` 키에 두 번째 값인 빗만 저장하는 것이죠. 그래서 `eng` 딕셔너리에 `brush` 키를 두 번 넣어도 저장된 값은 하나입니다.

```
print(eng)
```

```
실행결과
{'break': '깨어지다', 'bring': '가져오다', 'brush': '빗', 'brave': '용감한'}
```

실수로 딕셔너리의 키를 중복해서 작성하면 마지막 값만 남고 나머지 값은 사라질 수 있으니 유의하세요!

TIP 키는 중복할 수 없지만, 값은 여러 개를 넣을 수 있습니다. 다음처럼 리스트 형태로 넣으면 됩니다.

```
eng = {'break': ['깨어지다', '부서지다'], 'bring': '가져오다', 'brush': ['붓', '빗']}
print(eng['break'])
```

실행결과

```
['깨어지다', '부서지다']
```

TIP 단, 리스트는 값에만 넣을 수 있고 키에는 넣을 수 없습니다.

그럼 딕셔너리를 어떻게 만들어 사용하는지 알아봅시다. 분식집 메뉴판을 딕셔너리로 만들어 보겠습니다. 손님이 메뉴를 입력하면 가격을 출력하는 프로그램을 작성해 봅시다. 메뉴판에 들어갈 메뉴와 가격은 김밥은 2,500원, 참치김밥은 3,500원, 라면은 3,000원, 치즈라면은 4,000원입니다. 먼저 딕셔너리로 저장하려면 키와 값이 대응해야 합니다. 메뉴와 가격 중 어느 것이 딕셔너리의 키가 돼야 할까요? 앞에서 메뉴를 입력했을 때 가격을 출력한다고 했으니 메뉴를 기준으로 가격을 찾아야겠죠. 따라서 메뉴가 키가 돼야 합니다.

```
menu = {'김밥': 2500, '참치김밥': 3500, '라면': 3000, '치즈라면': 4000}
eat = input('어떤 메뉴를 드시겠습니까? ')
print(menu[eat], '원입니다.')
```

```
실행결과
어떤 메뉴를 드시겠습니까? 참치김밥
3500 원입니다.
```

메뉴를 키로, 가격을 값으로 하는 `menu` 딕셔너리를 만듭니다. `input()` 명령어로 메뉴를 입력받으면 이를 `eat` 변수에 저장합니다. 이때 입력받는 메뉴가 딕셔너리의 키이므로 `menu[eat]`으로 입력받은 메뉴의 가격을 찾아 출력합니다.

어떤가요? 리스트를 이해했다면 딕셔너리도 이해하기 어렵지 않을 겁니다. 리스트의 숫자 인덱스가 문자로 바뀐 것일 뿐이에요.

7.1.3 데이터 변경, 추가, 삭제하기

지금까지는 딕셔너리를 만들고 출력하기만 했습니다. 이번에는 딕셔너리의 값을 변경하는 방법과 데이터를 추가하고 삭제하는 방법을 배워 보겠습니다.

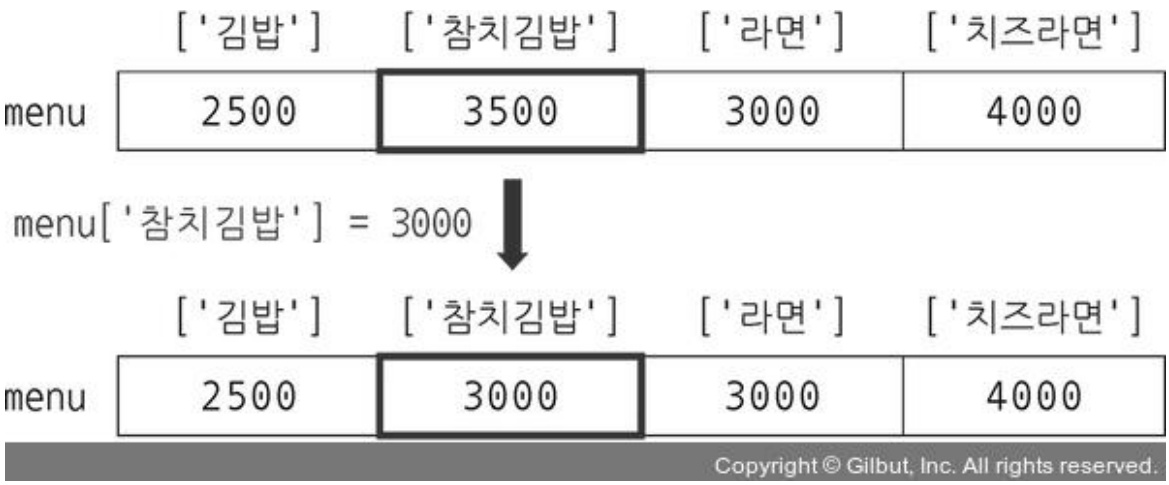
값 변경하기

값을 변경할 때는 키에 새로운 값을 넣으면 됩니다. 방법은 변수에 값을 저장할 때와 비슷합니다.

형식

```
딕셔너리명[키] = 값
```

예를 들어, 참치김밥의 가격을 3,500원에서 3,000원으로 내리려고 합니다. `참치김밥` 키를 사용하면 값인 가격에 접근할 수 있죠. 따라서 `menu['참치김밥'] = 3000`이라고 작성하면 됩니다.



Copyright © Gilbut, Inc. All rights reserved.

이를 그대로 코드로 바꿔 봅시다.

```
menu = {'김밥': 2500, '참치김밥': 3500, '라면': 3000, '치즈라면': 4000}
menu['참치김밥'] = 3000
print(menu)
```

```
실행결과
{'김밥': 2500, '참치김밥': 3000, '라면': 3000, '치즈라면': 4000}
```

`menu` 딕셔너리를 처음 만들 때는 `참치김밥` 키의 값은 `3500`이었습니다. `menu['참치김밥'] = 3000`을 실행하니 값이 `3000`으로 바뀌었습니다. 이처럼 딕셔너리의 값을 변경하는 것은 사실 변수의 값을 변경하는 것과 비슷합니다.

다른 메뉴의 가격도 변경해 봅시다. 라면은 2,500원으로, 치즈라면은 3,500원으로 변경하고 `menu` 딕셔너리를 출력합니다.

```
menu['라면'] = 2500
print(menu)
menu['치즈라면'] = 3500
print(menu)
```

```
실행결과
{'김밥': 2500, '참치김밥': 3000, '라면': 2500, '치즈라면': 4000}
{'김밥': 2500, '참치김밥': 3000, '라면': 2500, '치즈라면': 3500}
```

라면의 가격을 바꾸려면 `menu` 딕셔너리에서 `라면` 키에 새로운 값을 저장하면 됩니다. `menu['라면'] = 2500`을 실행하니 기존 값인 `3000`이 사라지고 `2500`이 저장됩니다. 같은 방법으로 `menu['치즈라면'] = 3500`을 실행하면 `치즈라면` 키에 값이 `3500`으로 저장됩니다.

이번에는 값으로 연산해 봅시다. 가격을 올릴 때 지금처럼 새로운 값을 저장할 수도 있지만, 기존 가격에 인상 금액만 더하는 방법도 있습니다. 딕셔너리의 값을 연산하는 방법 역시 변수와 비슷합니다. `변수 += 숫자`로 기존 변수 값을 증가시켰듯이 `딕셔너리[키] += 숫자` 형식으로 값에 더하기를 할 수 있습니다. 더하기뿐만 아니라 `-=`, `*=`, `/=` 등 다른 연산도 할 수 있습니다.

이 방법으로 김밥의 가격을 500원 올려 봅시다.

```
menu = {'김밥': 2500, '참치김밥': 3500, '라면': 3000, '치즈라면': 4000}
menu['김밥'] += 500
print(menu)
```

```
실행결과
{'김밥': 3000, '참치김밥': 3500, '라면': 3000, '치즈라면': 4000}
```

이번에는 참치김밥의 가격을 800원 인하합니다.

```
menu['참치김밥'] -= 800
print(menu)
```

```
실행결과
{'김밥': 3000, '참치김밥': 2700, '라면': 3000, '치즈라면': 4000}
```

이처럼 키로 값을 변경할 수 있습니다.

키-값 추가하기

이번에는 딕셔너리에 새로운 데이터를 추가해 봅시다. 데이터를 추가하는 것은 딕셔너리를 한 칸 더 늘리는 작업이라고 보면 됩니다. 새로운 키와 값 한 쌍을 넣어 주면 한 칸이 추가됩니다. 예를 들어, 가격이 6,000원인 돈가스를 추가하려면 키와 값을 대응해 `menu['돈가스'] = 6000`을 실행합니다. 그러면 `menu` 딕셔너리의 마지막 칸에 `돈가스(키)-6000(값)` 쌍이 추가됩니다.



코드로도 확인해 봅시다.

```
menu = {'김밥': 2500, '참치김밥': 3500, '라면': 3000, '치즈라면': 4000}
menu['돈가스'] = 6000
print(menu)
```

```
실행결과
{'김밥': 2500, '참치김밥': 3500, '라면': 3000, '치즈라면': 4000, '돈가스': 6000}
```

딕셔너리에 `'돈가스': 6000` 쌍이 추가됐습니다.

이번에는 추가한 돈가스의 가격을 6,500원으로 변경해 보겠습니다.

```
menu['돈가스'] = 6500
print(menu)
```

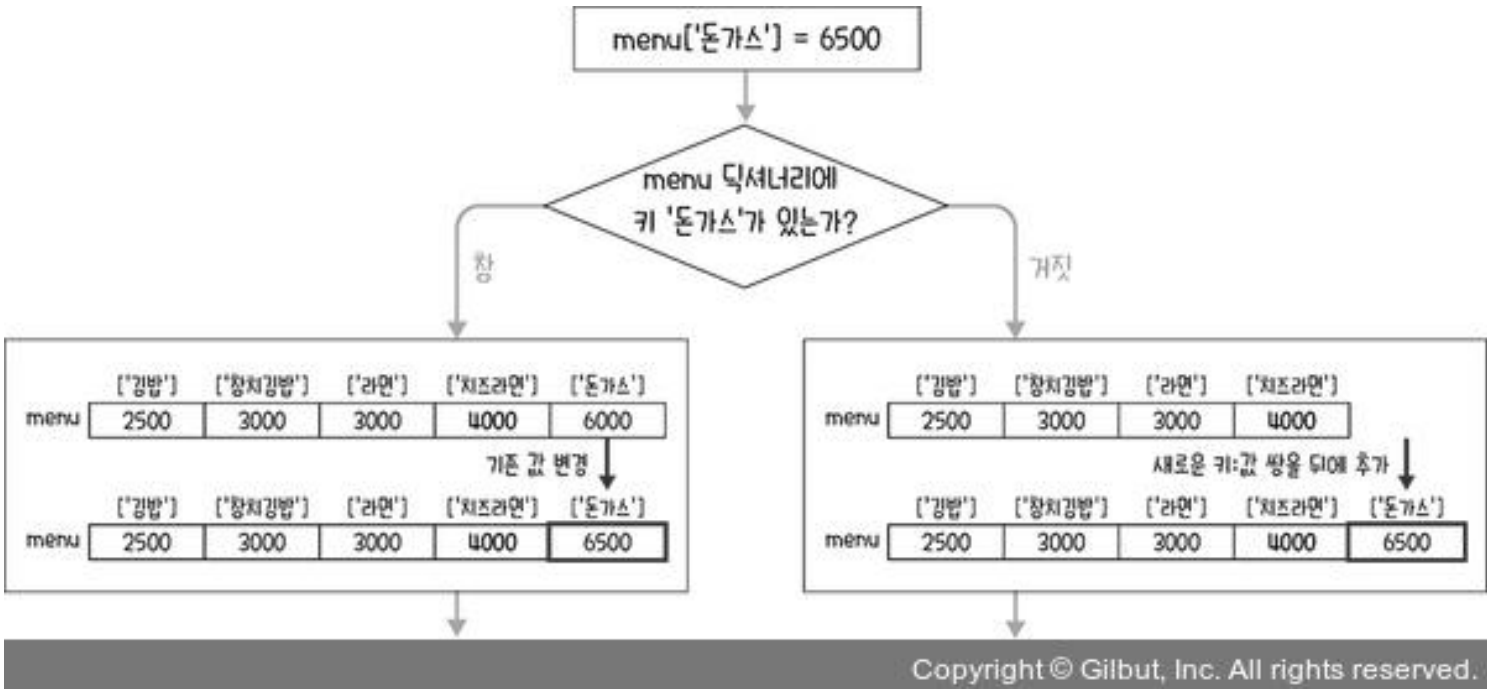
```
실행결과
{'김밥': 2500, '참치김밥': 3500, '라면': 3000, '치즈라면': 4000, '돈가스': 6500}
```

`돈가스` 키의 값이 `6500`으로 변경됩니다. 코드를 보면 알 수 있듯이 기존 값을 변경하는 것과 새로운 데이터를 추가하는 방법이 같습니다.

형식

```
딕셔너리명[키] = 값
```

그래서 기존에 키가 있는지 없는지 확인하고 값을 변경하거나 데이터를 추가해야 합니다. 기존에 같은 키가 있다면 값을 변경하고, 같은 키가 없다면 새로운 데이터를 추가합니다.



▲ 그림 7-1 프로그램 흐름

빈 딕셔너리를 만들어 데이터를 새로 추가할 수도 있습니다. 빈 리스트를 만든 후 데이터를 추가했듯이 딕셔너리도 `딕셔너리명 = {}` 형식으로 빈 딕셔너리를 만들고 데이터를 추가합니다.

```
eng = {}
print(eng)
eng['강아지'] = 'dog'
print(eng)
eng['고양이'] = 'cat'
print(eng)
```

```
실행결과
{}
{'강아지': 'dog'}
{'강아지': 'dog', '고양이': 'cat'}
```

처음에 데이터가 없는 빈 딕셔너리 `eng`를 만들면 이 상태에서는 `{}`만 출력됩니다. 첫 번째 데이터를 `eng['강아지'] = 'dog'`로 추가하고, 두 번째 데이터를 `eng['고양이'] = 'cat'`으로 추가했습니다.

이번에는 신메뉴 이름과 가격을 입력받아 분식집 메뉴판에 추가하는 프로그램을 작성해 봅시다. 데이터를 넣어 `menu` 딕셔너리를 만들고 새로운 메뉴와 가격을 `input()` 명령어로 입력받아 추가합니다.

코드를 보기 전에 직접 작성하고 책의 코드를 보세요! 필요한 기능은 다 배웠으니 조금만 생각하면 작성할 수 있습니다.

```
menu = {'김밥': 2500, '참치김밥': 3500, '라면': 3000, '치즈라면': 4000}
name = input('추가할 메뉴는? ')
price = int(input('메뉴의 가격은? '))
menu[name] = price
print(menu)
```

```
실행결과
추가할 메뉴는? 쫄면
메뉴의 가격은? 4000
{'김밥': 2500, '참치김밥': 3500, '라면': 3000, '치즈라면': 4000, '쫄면': 4000}
```

메뉴는 `name`, 가격은 `price` 변수로 각각 입력받습니다. 이때 가격은 숫자이므로 `int()`로 감싸 줘야 합니다. 그런 다음 입력받은 메뉴와 가격을 `menu` 딕셔너리에 새로운 키와 값으로 추가합니다. 새로운 키는 메뉴가 담긴 `name` 변수, 새로운 값은 가격이 담긴 `price` 변수이므로 `menu[name] = price`로 작성하면 됩니다. '쫄면'과 '4000'을 입력하면 `menu` 딕셔너리의 가장 마지막에 '쫄면: 4000' 데이터가 추가됩니다.

키-값 삭제하기

마지막으로 딕셔너리에서 키-값 쌍을 삭제하는 방법을 알아봅시다. 리스트에서 데이터를 삭제할 때 `del`을 사용했습니다. 딕셔너리에서도 `del`을 사용합니다.

형식

```
del 딕셔너리명[키]
```

`del` 뒤에 삭제할 키를 적어주면 해당 키-값 쌍이 삭제됩니다.

```
menu = {'김밥': 2500, '참치김밥': 3500, '라면': 3000, '치즈라면': 4000}
del menu['치즈라면']
print(menu)
```

실행결과

```
{'김밥': 2500, '참치김밥': 3500, '라면': 3000}
```

7.1.4 다양한 명령어 사용하기

앞에서 딕셔너리의 키와 값이 의미하는 바를 배웠습니다. 그렇다면 딕셔너리의 키나 값만 알고 싶을 때는 어떻게 할까요? 딕셔너리가 가진 명령어를 활용하면 됩니다. 이번에는 딕셔너리에서 데이터에 접근할 때 사용할 수 있는 다양한 명령어를 배워 보겠습니다.

딕셔너리의 키만 꺼내기: keys()

마트에서 파는 물건을 키로, 가격을 값으로 하는 딕셔너리를 만들어 봅시다. 딕셔너리명은 `mart`로 하고, 과자, 아이스크림, 휴지, 고기, 우유의 가격을 다음과 같이 저장합니다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
```

이 딕셔너리의 구조는 다음과 같습니다.



여기에서 가격은 제외하고 어떤 물건을 파는지만 확인하고 싶다고 해 봅시다. `mart` 딕셔너리의 키만 뽑아내면 되는데, 이럴 때는 `keys()` 명령어를 사용합니다. 실행할 때는 딕셔너리명 뒤에 도트(`.`)를 넣고 `keys()` 명령어를 붙이면 됩니다.

형식

```
딕셔너리명.keys()
```

`mart` 딕셔너리의 키만 뽑아낼 때는 `mart.keys()`를 실행하면 됩니다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(mart.keys())
```

```
실행결과
dict_keys(['과자', '아이스크림', '휴지', '고기', '우유'])
```

`mart` 딕셔너리의 키인 물건 이름만 출력합니다. 실행결과의 가장 앞에 나온 `dict_keys()`는 딕셔너리의 키임을 나타내는 데이터 타입으로, `keys()`를 실행하면 결과는 자동으로 해당 데이터 타입이 됩니다. `type(mart.keys())`를 실행해 보면 `dict_keys`가 출력되는 것을 볼 수 있습니다.

```
type(mart.keys())
```

```
실행결과
dict_keys
```

이처럼 `keys()`를 사용해 간단하게 키만 추출할 수 있습니다.

딕셔너리의 값만 꺼내기: values()

이번에는 키가 아닌 값만 출력해 볼까요? 값만 선택하고 싶으면 `keys()`와 마찬가지로 `values()`를 딕셔너리명 뒤에 붙여 실행하면 됩니다.

형식

```
딕셔너리명.values()
```

`mart` 딕셔너리의 값만 출력해 봅시다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(mart.values())
```

실행결과

```
dict_values([1500, 1700, 4800, 10500, 2600])
```

딕셔너리의 값인 물건의 가격만 출력합니다. 그리고 `mart.keys()` 때처럼 결과 앞에 딕셔너리의 값임을 나타내는 데이터 타입 `dict_values()`가 나옵니다.

NOTE 추출한 결괏값을 리스트로 바꾸기

`dict_keys`와 `dict_values` 데이터 타입은 리스트로 바꿀 수 있습니다. 데이터 타입을 변환하는 방법은 3장 변수에서 배웠듯이 바꾸고 싶은 데이터 타입으로 바꿀 대상을 감싸는 것입니다. 리스트로 변환하는 방법도 같습니다. 변환할 대상을 `list()`로 감싸면 됩니다. 앞의 `dict_keys`를 리스트로 바꿔 출력해 봅시다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(list(mart.keys()))
```

```
실행결과
['과자', '아이스크림', '휴지', '고기', '우유']
```

`list()`로 감싸니 대괄호로 둘러 쌓인 리스트 타입으로 변경됐습니다. `dict_values`도 리스트로 바꾸어 출력해 봅시다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(list(mart.values()))
```

```
실행결과
[1500, 1700, 4800, 10500, 2600]
```

값만 추출한 결과도 리스트로 잘 변환됩니다. 원하는 데이터 타입으로 바꿀 수 있으니 보기 편한 쪽으로 출력하세요.

딕셔너리의 키-값 꺼내기: items()
키나 값만이 아니라 키-값을 한 쌍으로 출력하는 방법도 있습니다. 형식은 앞에서와 같고 딕셔너리명 뒤에 명령어만 `items()` 로 바꿉니다.

형식

```
딕셔너리명.items()
```

`mart` 딕셔너리의 키-값을 출력해 봅시다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(mart.items())
```

실행결과

```
dict_items([('과자', 1500), ('아이스크림', 1700), ('휴지', 4800), ('고기', 10500), ('우유', 2600)])
```

딕셔너리의 키-값이 한 쌍씩 소괄호 안에 들어가 출력됩니다. `items()` 역시 제어구조에서 유용하게 쓰입니다.

딕셔너리 정렬하기: sorted()

이번에는 딕셔너리의 데이터를 정렬해 보겠습니다. 정렬은 오름차순이나 내림차순으로 데이터를 나열하는 것이죠. 딕셔너리를 정렬할 때는 `sorted()` 명령어를 사용합니다. 그런데 딕셔너리에는 키와 값 두 가지 요소가 있기 때문에 어떤 걸 기준으로 정렬할지를 명시해야 합니다. 이럴 때 앞에서 배운 `keys()`, `values()`, `items()` 명령어를 활용합니다.

`mart` 딕셔너리를 정렬해 봅시다. 먼저 키를 기준으로 정렬하려면 `sorted()` 안에 `mart.keys()`를 넣어 `sorted(mart.keys())`를 실행합니다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(sorted(mart.keys()))
```

```
실행결과
['고기', '과자', '아이스크림', '우유', '휴지']
```

`mart` 딕셔너리의 키만 추출해 오름차순으로 정렬합니다. 오름차순은 숫자는 작은 것부터, 문자는 가나다순이나 알파벳순으로 정렬하는 방법입니다. 역순, 즉 내림차순으로 정렬하고 싶으면 리스트처럼 `reverse=True` 옵션을 추가합니다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(sorted(mart.keys(), reverse=True))
```

```
실행결과
['휴지', '우유', '아이스크림', '과자', '고기']
```

휴지가 첫 번째에, 고기가 마지막에 배치됩니다. 오름차순이나 내림차순으로 정렬한 두 코드의 실행결과가 모두 대괄호(`[]`)로 감싸져 있죠. sorted()로 딕셔너리를 정렬하면 결과가 리스트로 나온다는 뜻입니다. 이런 결과는 데이터를 다룰 때 유용합니다. 예를 들어, `sorted(mart.keys())`를 실행하면 다음처럼 리스트로 정렬됩니다.



Copyright © Gilbut, Inc. All rights reserved.

리스트이므로 숫자 인덱스가 생기죠. 따라서 리스트의 유용한 기능인 슬라이싱도 할 수 있습니다. 리스트에서 처음부터 두 번째 요소까지 추출하려면 `[:2]`를 붙입니다. `sorted(mart.keys())` 자체가 하나의 리스트이므로 `sorted(mart.keys())` 바로 뒤에 `[:2]`를 붙이면 됩니다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(sorted(mart.keys())[:2])
```

```
실행결과
['고기', '과자']
```

이 기능은 딕셔너리의 값을 기준으로 정렬할 때 더 유용합니다. 딕셔너리의 값을 정렬하는 방법은 키를 정렬할 때와 같고 `keys()`만 `values()`로 바꾸어 `sorted(딕셔너리명.values())`를 실행하면 됩니다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(sorted(mart.values()))
```

```
실행결과
[1500, 1700, 2600, 4800, 10500]
```

값을 기준으로 오름차순 정렬된 결과가 나옵니다.

이번에는 높은 가격부터 낮은 가격 순서로 정렬해 보겠습니다. 앞의 코드에 내림차순 옵션인 `reverse=True`를 적용하면 됩니다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(sorted(mart.values()), reverse=True))
```

```
실행결과
[10500, 4800, 2600, 1700, 1500]
```

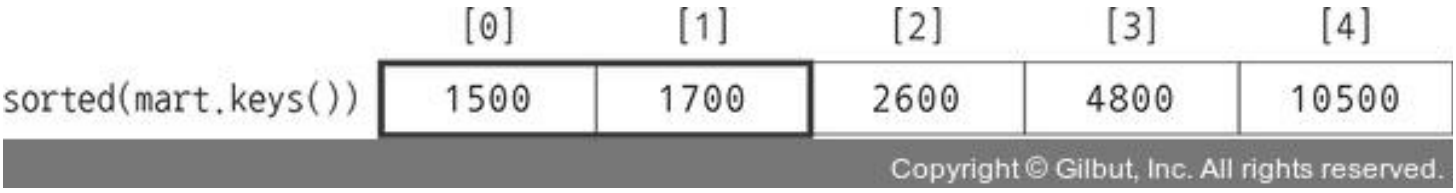
높은 가격부터 낮은 가격순으로 정렬합니다.

이번에는 `mart` 딕셔너리에서 가장 낮은 가격 2개를 뽑아 봅시다. `mart`의 값들을 오름차순 정렬한 후 가장 앞에 있는 요소 2개만 뽑아내면 되겠죠?

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(sorted(mart.values())[:2])
```

```
실행결과
[1500, 1700]
```

`sorted(mart.values())` 자체가 리스트이므로 바로 뒤에 `[:2]`를 붙여 출력하면 리스트의 가장 앞에 있는 두 요소만 결과로 볼 수 있습니다.



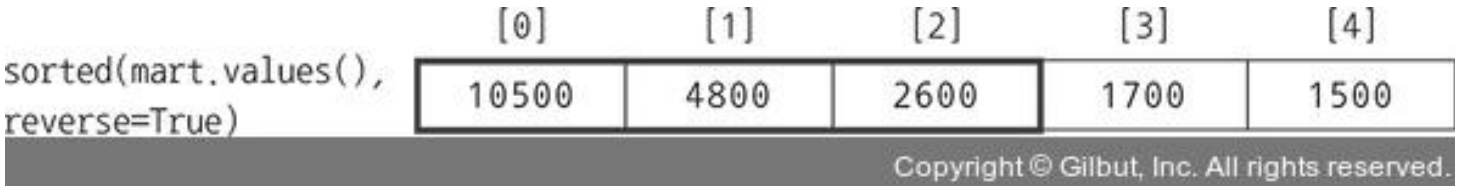
Copyright © Gilbut, Inc. All rights reserved.

이번에는 `mart`에서 가장 높은 가격 3개를 뽑아 봅시다. 가격을 내림차순 정렬한 후 리스트의 가장 앞에 있는 요소 3개만 출력하면 됩니다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(sorted(mart.values(), reverse=True)[:3])
```

```
실행결과
[10500, 4800, 2600]
```

`reverse=True` 옵션을 붙여 `sorted(mart.values())` 명령어를 실행한 후 `[:3]`으로 슬라이싱하니 가장 앞에 있는 요소 3개만 결과로 볼 수 있습니다(닫는 괄호의 위치에 유의하세요).



Copyright © Gilbut, Inc. All rights reserved.

이처럼 `sorted()`를 사용해 키나 값을 기준으로 딕셔너리를 정렬하거나 원하는 값을 뽑아낼 수 있습니다.

NOTE sorted() 정렬의 기본값

딕셔너리를 정렬할 때 `sorted(딕셔너리명)`만 실행하면 자동으로 키를 기준으로 정렬합니다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(sorted(mart))
```

실행결과

```
['고기', '과자', '아이스크림', '우유', '휴지']
```

그럼 딕셔너리의 키와 값을 한 쌍으로 묶어서 정렬할 수는 없을까요? 예를 들어, 마트에서 가장 싼 물건과 그 가격을 알고 싶을 때는 어떻게 할까요? 이때는 딕셔너리의 키-값을 동시에 꺼낼 때 쓴 `items()`를 사용하면 됩니다.

형식

```
sorted(딕셔너리명.items())
```

`mart` 딕셔너리에 적용해 봅시다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(sorted(mart.items()))
```

실행결과

```
[('고기', 10500), ('과자', 1500), ('아이스크림', 1700), ('우유', 2600), ('휴지', 4800)]
```

키와 값이 소괄호 안에 한 쌍으로 묶여 출력됩니다. 이때 정렬 기준은 무엇일까요? 키와 값을 비교해 보면 키를 기준으로 오름차순 정렬됐음을 알 수 있습니다.

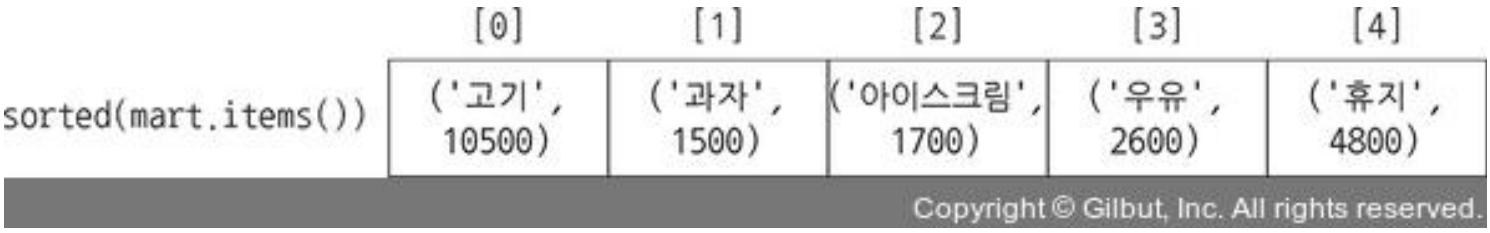
키를 기준으로 오름차순 정렬되니 `sorted(딕셔너리명.items(), reverse=True)`를 실행하면 키를 기준으로 내림차순 정렬되겠죠? 한번 실행해 봅시다.

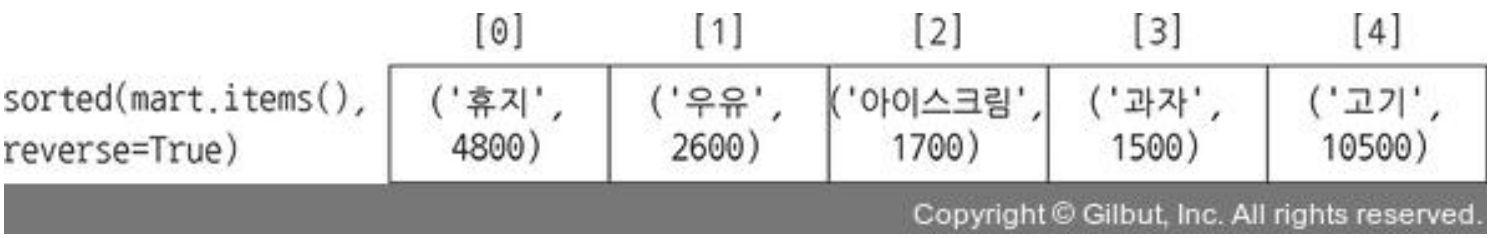
```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(sorted(mart.items(), reverse=True))
```

실행결과

```
[('휴지', 4800), ('우유', 2600), ('아이스크림', 1700), ('과자', 1500), ('고기', 10500)]
```

결과에서 키와 값이 한 쌍으로 묶여 있어서 딕셔너리로 보이지만, `items()`로 정렬한 결과도 리스트입니다. (키, 값) 한 쌍이 리스트 한 칸에 요소로 들어간 형태죠.





이번에는 물건을 저렴한 순서대로 정렬해서 어떤 물건이 싼지 알고 싶습니다. 이때는 키가 아닌 값을 기준으로 정렬해야 합니다. 값으로 정렬하려면 `operator`라는 모듈이 필요합니다. `operator`에는 계산을 도와주는 다양한 명령어가 포함되어 있습니다. 여기서는 `operator`에 포함된 `itemgetter()`를 사용합니다. `itemgetter()`는 기준에 따라 키와 값을 모으는 명령어입니다. 우선 다음처럼 코드를 작성하고 실행해 봅시다.

```
import operator

mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(sorted(mart.items(), key=operator.itemgetter(0)))
```

```
실행결과
[('고기', 10500), ('과자', 1500), ('아이스크림', 1700), ('우유', 2600), ('휴지', 4800)]
```

모듈도 라이브러리처럼 `import`로 추가합니다. `sorted()`에 추가된 `key`는 정렬 기준을 나타내는 옵션입니다. 여기서는 `operator`의 `itemgetter()`를 정렬 기준으로 사용한단 의미로 `operator.itemgetter()`를 값으로 넣습니다. 그리고 소괄호 안의 숫자는 키 또는 값을 표시하는 숫자입니다. 0이면 키를 기준으로, 1이면 값을 기준으로 정렬합니다.

```
import operator

mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(sorted(mart.items(), key=operator.itemgetter(1)))
```

```
실행결과
[('과자', 1500), ('아이스크림', 1700), ('우유', 2600), ('휴지', 4800), ('고기', 10500)]
```

`key=operator.itemgetter(1)` 옵션을 주니 값을 기준으로 정렬합니다.

이번에는 높은 가격부터 낮은 가격 순서로 물건과 가격을 정렬해 봅시다. 값을 기준으로 정렬하고 내림차순 옵션만 추가하면 됩니다.

```
import operator

mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(sorted(mart.items(), key=operator.itemgetter(1), reverse=True))
```

```
실행결과
[('고기', 10500), ('휴지', 4800), ('우유', 2600), ('아이스크림', 1700), ('과자', 1500)]
```

가장 비싼 고기부터 가장 싼 과자까지 가격순으로 정렬합니다.

그러면 더 나가서 가장 저렴한 물건 두 가지와 그 가격을 출력해 봅시다. 책의 코드를 확인하기 전에 앞에서 배운 내용을 참고해 직접 작성해 보세요!

```
import operator

mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
print(sorted(mart.items(), key=operator.itemgetter(1))[:2])
```

```
실행결과
[('과자', 1500), ('아이스크림', 1700)]
```

`sorted()` 결과가 리스트이므로 이 뒤에 `[:2]`를 붙여 슬라이싱합니다. 가장 저렴한 물건 세 가지를 알고 싶다면 `[:3]`을 붙이면 되겠죠? 딕셔너리를 정렬하는 기능은 데이터 분석 프로젝트에서 많이 쓰니 잘 알아 두기 바랍니다.

NOTE 딕셔너리를 정렬해도 원본 딕셔너리는 영향을 받지 않는다?

이 말이 무슨 뜻일까요? 코드를 살펴봅시다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
sorted(mart.keys())
print(mart)
```

실행결과

```
{'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
```

`mart` 딕셔너리를 만든 다음 키를 기준으로 딕셔너리를 정렬했습니다. 그러고 나서 `mart` 딕셔너리를 출력해 보니 정렬되지 않은 처음 상태 그대로 출력됩니다. 이처럼 `sorted()`는 딕셔너리를 정렬해서 보여 줄 뿐, 딕셔너리에 저장된 내용에는 영향을 주지 않습니다. 정렬한 결과는 다음처럼 원래 딕셔너리와는 별개입니다. 정렬한 결과를 기억하고 싶으면 변수에 저장해야 합니다.

정렬한 결과를 원래 딕셔너리가 저장되어 있던 `mart`에 다시 저장해 봅시다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
mart = sorted(mart.keys())
print(mart)
```

실행결과

```
['고기', '과자', '아이스크림', '우유', '휴지']
```



`sorted()`의 결과는 리스트이기 때문에 `mart = sorted(mart.items())`를 실행하면 `mart`는 딕셔너리가 아닌 리스트가 됩니다. 코드 실행 후 키가 사라지고 인덱스가 생긴 걸 확인할 수 있습니다.

정렬된 결과를 딕셔너리로 저장하려면 `dict()`로 감싸 데이터 타입을 딕셔너리로 변환하면 됩니다. 단, 딕셔너리는 키와 값이 필요하므로 `items()`로 정렬한 결과만 딕셔너리로 변환할 수 있습니다.

```
mart = {'과자': 1500, '아이스크림': 1700, '휴지': 4800, '고기': 10500, '우유': 2600}
mart = dict(sorted(mart.items()))
print(mart)
```

```
실행결과
{'고기': 10500, '과자': 1500, '아이스크림': 1700, '우유': 2600, '휴지': 4800}
```

키-값 쌍으로 딕셔너리를 정렬하고, 이를 `dict()`로 감싸 딕셔너리 타입으로 다시 저장합니다.

1분 퀴즈

1 다음 코드는 영상의 주제별 시청 횟수를 딕셔너리로 저장한 것입니다. 다음 중 옳지 않은 것을 고르세요.

```
history = {'강아지': 290, '먹방': 194, '요리': 491, '뉴스': 85, '다이어트': 362}
```

① '먹방' 시청 횟수를 출력하려면 `print(history['먹방'])`을 실행한다.

② 새로운 주제인 '고양이' 영상을 시청했다면 `history['고양이'] += 1`을 실행한다.

③ 뉴스 시청 기록을 삭제하려면 `del history['뉴스']`를 실행한다.

④ `print(history.values())`를 실행하면 시청 횟수만 출력된다.

⑤ 영상을 많이 시청한 횟수순으로 정렬하려면 `import operator`를 추가한 후, `print(sorted(history.items(), reverse=True, key=operator.itemgetter(1)))`을 실행한다.

정답 및 해설

7.2

제어구조로 딕셔너리 다루기

이번에는 조건문과 반복문을 사용해 딕셔너리를 다뤄 보겠습니다.

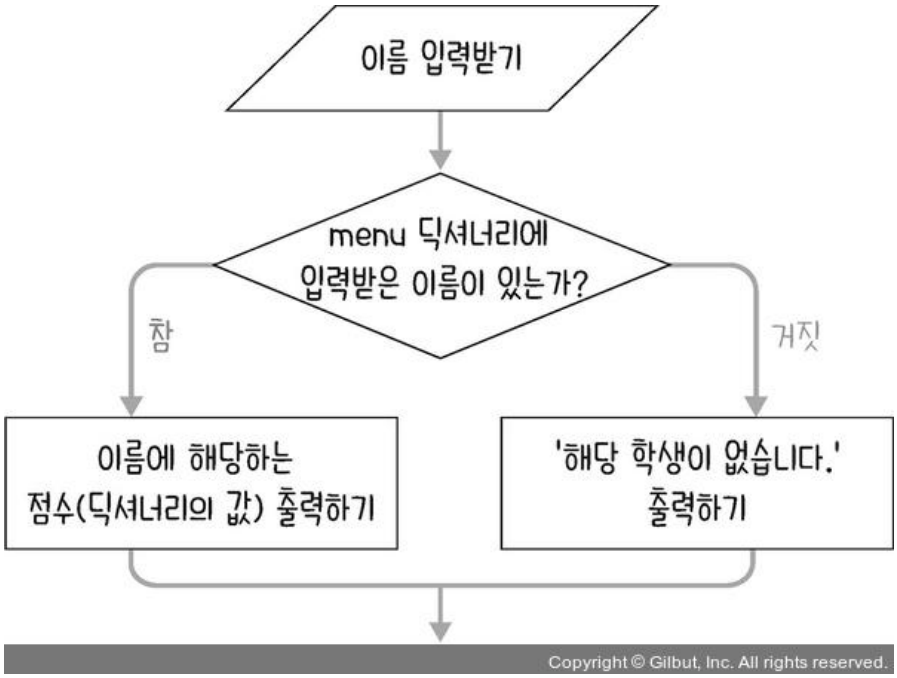
7.2.1 조건문으로 딕셔너리 다루기

다음과 같이 학생 다섯 명의 중간고사 점수를 담은 딕셔너리를 만듭니다.

```
midterm = {'도윤': 43, '하윤': 82, '시우': 76, '지유': 61, '주원': 94}
```

딕셔너리명은 `midterm`이며, 학생 이름을 키로, 중간고사 점수를 값으로 넣었습니다. 학생 이름을 입력받아서 학생 이름이 `midterm` 딕셔너리에 있으면 점수를 출력하고, 딕셔너리에 없으면 '해당 학생이 없습니다.'라고 출력해 봅시다.

코딩하기 전에 먼저 생각해 볼까요? 딕셔너리에 학생 이름이 있는 경우와 없는 경우를 기준으로 실행해야 하는 명령이 달라집니다. 따라서 조건문을 활용하면 됩니다. 조건은 뭐가 될까요? 입력받은 학생 이름이 `midterm` 딕셔너리의 키에 있는가 또는 없는가입니다. 프로그램의 흐름을 정리하면 다음과 같습니다.



▲ 그림 7-2 프로그램 흐름

프로그램의 흐름에 맞춰 코드를 작성해 봅시다.

```
midterm = {'도윤': 43, '하윤': 82, '시우': 76, '지유': 61, '주원': 94}
student = input('어떤 학생의 점수가 궁금한가요? ')
if student in midterm.keys():
    print(midterm[student], '점입니다.')
else:
    print('해당 학생이 없습니다.')
```

```
실행결과
키가 있을 때
어떤 학생의 점수가 궁금한가요? 하윤
82 점입니다.

키가 없을 때
어떤 학생의 점수가 궁금한가요? 서진
해당 학생이 없습니다.
```

입력된 학생 이름을 `student` 변수에 저장하고, 저장된 이름이 딕셔너리의 키(`midterm.keys()`)에 있다면 해당 키의 값(`midterm[student]`)을 출력합니다. 저장된 이름이 키에 없다면 '해당 학생이 없습니다.'를 출력합니다. 딕셔너리에 해당 키가 있는지 없는지를 확인하므로 `in` 키워드를 활용해 조건은 `student in midterm.keys()`로 작성합니다. 이처럼 조건문을 사용해 딕셔너리에서 조건을 충족하는 값만 선택할 수 있습니다.

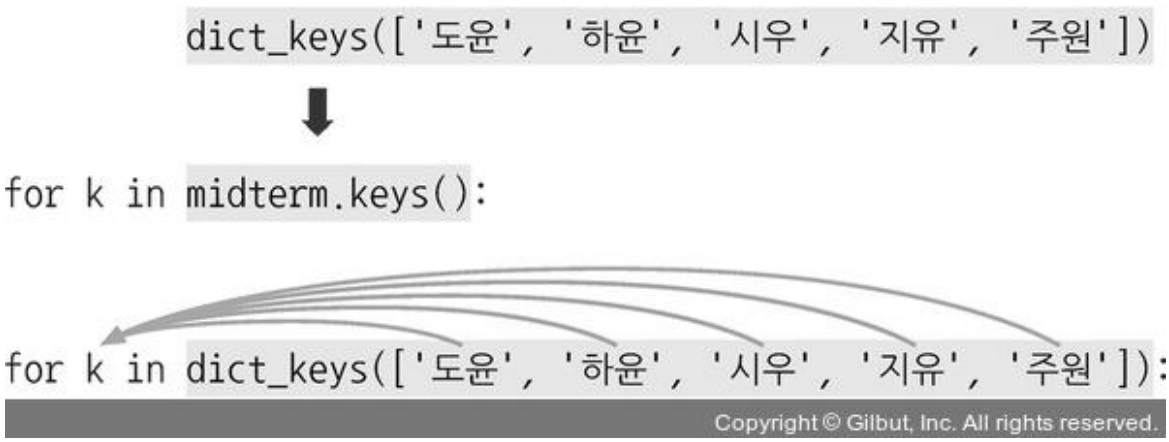
7.2.2 반복문으로 딕셔너리 다루기

이번에는 반복문을 사용해 보겠습니다. `midterm` 딕셔너리에서 학생 이름을 반복문으로 출력해 봅시다. 반복문의 형식은 `for i in data`입니다. `data` 위치에 무엇이 들어갈까요? 먼저 딕셔너리의 키를 넣어 보겠습니다.

```
midterm = {'도윤': 43, '하윤': 82, '시우': 76, '지유': 61, '주원': 94}
for k in midterm.keys():
    print(k, end=' ')
```

```
실행결과
도윤 하윤 시우 지유 주원
```

`midterm.keys()`는 `dict_keys(['도윤', '하윤', '시우', '지유', '주원'])`이므로 다음처럼 `data` 위치에 `midterm` 딕셔너리의 키만 추출한 리스트가 들어갑니다. 그리고 이 키가 차례대로 변수 `k`에 저장되죠(데이터가 딕셔너리의 키라서 변수는 `k`를 사용합니다).



Copyright © Gilbut, Inc. All rights reserved.

이번에는 점수만 출력해 봅시다. `data` 위치에 딕셔너리의 값을 추출해 넣으면 되니 `midterm.keys()`를 `midterm.values()`로 바꾸면 되겠죠? 실행해 봅시다.

```
midterm = {'도윤': 43, '하윤': 82, '시우': 76, '지유': 61, '주원': 94}
for v in midterm.values():
    print(v, end=' ')
```

```
실행결과
43 82 76 61 94
```

`midterm` 딕셔너리의 값, 즉 학생들의 점수만 출력됩니다. 여기서는 반복할 데이터 묶음이 딕셔너리의 값이어서 반복문의 변수를 `v`로 지정했습니다. 딕셔너리의 값을 차례대로 변수 `v`에 담고 값의 개수만큼 반복해서 `print()`를 실행합니다.

이번에는 키와 값을 같이 출력해 보겠습니다. 키와 값을 같이 출력하는 방법은 세 가지입니다. 먼저 키를 이용해 값을 출력하는 방법입니다. 반복문 `for k in midterm.keys()`에서 `midterm` 딕셔너리의 키가 `k`에 차례대로 담깁니다. 따라서 `k`에 대응하는 값은 `midterm[k]`로 얻을 수 있습니다.

```
midterm = {'도윤': 43, '하윤': 82, '시우': 76, '지유': 61, '주원': 94}
for k in midterm.keys():
    print(k, midterm[k], end=' ')
```

```
실행결과
도윤 43, 하윤 82, 시우 76, 지유 61, 주원 94
```

이번에는 `midterm.items()`로 키-값을 쌍으로 출력해 보겠습니다.

```
midterm = {'도윤': 43, '하윤': 82, '시우': 76, '지유': 61, '주원': 94}
for i in midterm.items():
    print(i, end=' ')
```

```
실행결과
('도윤', 43) ('하윤', 82) ('시우', 76) ('지유', 61) ('주원', 94)
```

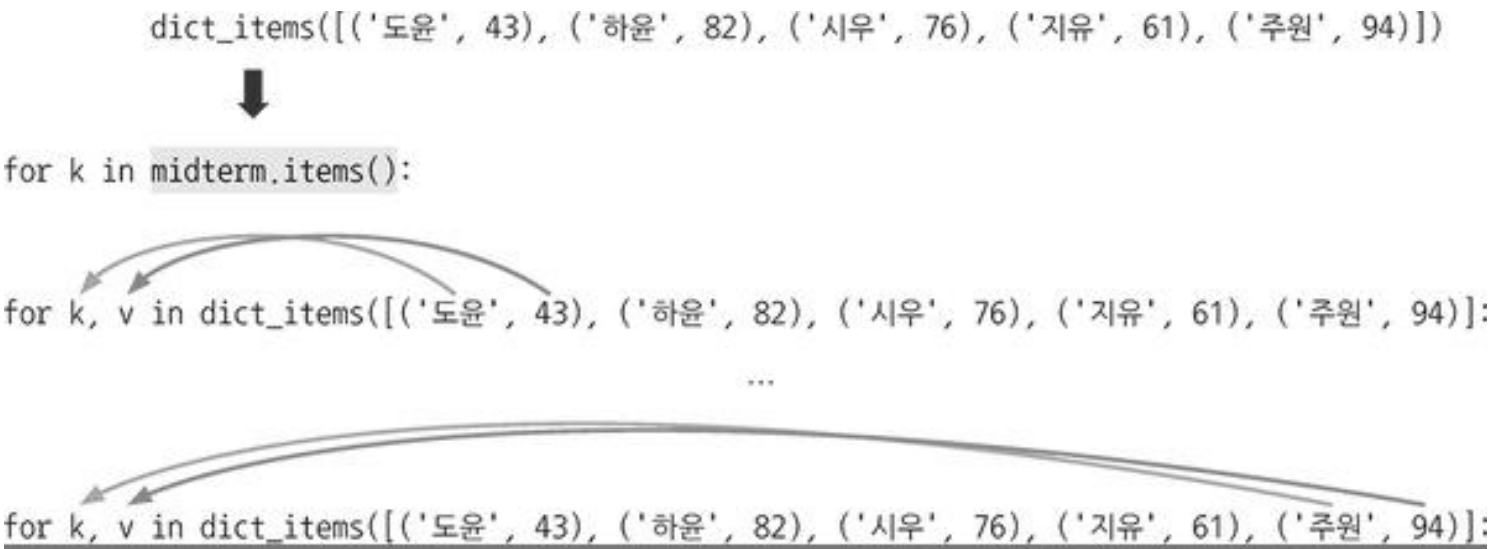
학생 이름과 중간고사 점수가 한 쌍씩 출력됩니다.

반복문에 `midterm.items()`를 사용할 때 키와 값을 각각 저장하는 방법도 있습니다. 변수를 2개 사용하면 자동으로 첫 번째 변수에 키를, 두 번째 변수에 값을 넣습니다. 코드로 확인해 봅시다.

```
midterm = {'도윤': 43, '하윤': 82, '시우': 76, '지유': 61, '주원': 94}
for k, v in midterm.items():
    print(k, v, end=' ')
```

```
실행결과
도윤 43 하윤 82 시우 76 지유 61 주원 94
```

반복문에서 변수를 `k`와 `v` 2개로 지정했습니다. 첫 번째 변수 `k`에는 `midterm` 딕셔너리의 키인 학생 이름이, 두 번째 변수 `v`에는 `midterm`의 값인 점수가 담겨 출력됩니다. 다음처럼 반복할 때마다 키-값이 한 쌍씩 변수 `k`와 `v`에 저장됩니다.



Copyright © Gilbut, Inc. All rights reserved.

7.2.3 중첩 제어구조로 딕셔너리 다루기

배운 내용을 응용해 봅시다. `midterm` 딕셔너리에서 점수가 70점 이상인 학생의 이름과 점수를 출력하고 싶습니다. 어떻게 작성할까요? 먼저 점수 70점 이상인 조건을 충족하는 경우를 골라내야 하므로 조건문이 필요합니다. 그리고 조건을 충족하는지 딕셔너리의 모든 요소를 확인해야 하므로 반복문이 필요하고 조건문은 반복문 내부에 있어야 합니다. 프로그램 흐름을 정리하면 다음과 같습니다.

```
딕셔너리의 모든 요소에서 반복
    만약 점수가 70 이상이라면
        이름, 점수 출력하기
```

프로그램의 흐름에 맞춰 코드를 작성해 봅시다.

```
midterm = {'도윤': 43, '하윤': 82, '시우': 76, '지유': 61, '주원': 94}
print('점수가 70점 이상인 학생의 이름과 점수를 모두 출력합니다.')
for k, v in midterm.items():
    if v >= 70:
        print(k, v, end=' ')
```

```
실행결과
점수가 70점 이상인 학생의 이름과 점수를 모두 출력합니다.
하윤 82 시우 76 주원 94
```

학생 이름과 점수를 출력해야 하므로 딕셔너리의 키와 값이 모두 필요합니다. 앞에서 반복문으로 키와 값을 출력하는 방법은 세 가지였습니다. 이 중에서 `midterm.items()`로 키와 값을 각각 저장하는 방법을 사용하겠습니다. 앞에서처럼 변수 2개를 사용하고, 첫 번째 변수 `k`에는 키가, 두 번째 변수 `v`에는 값이 자동으로 담깁니다. 그리고 조건문에서 `v`가 70 이상일 때만 `print()`를 실행해 70점 이상인 학생들의 이름과 점수를 출력합니다.

NOTE 반복문에 딕셔너리명만 사용한다면?

지금까지는 반복문에 범위를 딕셔너리의 키, 값, 키-값 쌍으로 직접 지정해 넣었습니다. 만약 딕셔너리명만 넣으면 어떻게 될까요?

```
midterm = {'도윤': 43, '하윤': 82, '시우': 76, '지유': 61, '주원': 94}
for m in midterm:
    print(m, end=' ')
```

```
실행결과
도윤 하윤 시우 지유 주원
```

딕셔너리의 이름만 넣으면 자동으로 딕셔너리의 키를 기준으로 반복문을 실행합니다. 딕셔너리를 정렬할 때도 `sorted()`에 딕셔너리명만 넣으면 키로 딕셔너리를 정렬했습니다. 딕셔너리에서 키는 기준 역할을 하기 때문에 기준을 따로 명시하지 않으면 대부분 키를 기준으로 사용합니다.

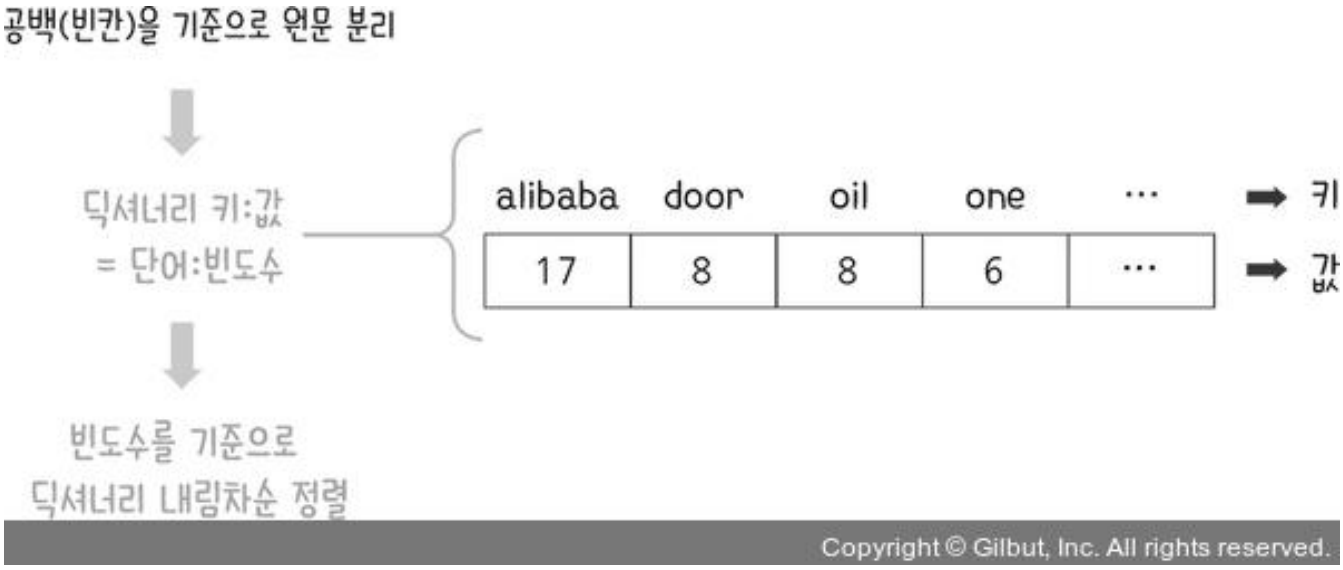
7.2.4 딕셔너리로 간단한 프로그램 만들기

딕셔너리를 활용해 영단어장을 만들어 봅시다. 영단어장은 빈도수를 기준으로 빈도수가 높은 단어부터 낮은 단어순으로 정렬합니다. 단어장은 『알리바바와 40인의 도적』[1]을 기반으로 만듭니다. 파일은 자료실에서 내려받을 수 있습니다. 원하는 글을 직접 넣어도 됩니다.

제공하는 파일은 원활한 실습을 위해 원문을 재편집했습니다. 영단어장을 만들 때 `split()`을 사용하므로 공백을 기준으로 단어가 분리되도록 문장 기호 양옆에는 공백을 넣었습니다. 그리고 대소문자를 구분하지 않으려고(Alibaba와 alibaba를 동일한 단어로 취급하도록) 모든 대문자를 소문자로 변경했습니다.

빈도순으로 단어장 만들기

단어장 프로그램의 흐름을 구상해 봅시다. 원문을 공백 기준으로 분리하므로 모든 문장이 단어로 쪼개집니다. 이 단어를 키로, 단어의 빈도수를 값으로 저장하는 딕셔너리를 만듭니다. 그리고 딕셔너리를 빈도수, 즉 값을 기준으로 내림차순 정렬하면 가장 자주 등장하는 단어부터 순서대로 정렬합니다.



▲ 그림 7-3 단어장 프로그램의 흐름

---

1 출처: https://www.gutenberg.org/files/37679/37679-h/37679-h.htm

원문은 `alibaba` 변수에, 단어와 빈도수는 `vocab` 딕셔너리에 저장합니다. 아직 넣을 단어가 없으므로 `vocab`은 빈 딕셔너리로 만듭니다. 그런 다음 원문을 저장한 `alibaba` 변수를 `split()`으로 분리합니다. `split()`을 실행하면 분리된 결과가 리스트에 담기는 것을 기억하죠? 분리된 결과를 담은 리스트를 `word_list`에 저장합니다.

```
alibaba = " in a town of persia lived two brothers , sons of a poor man; one named cassim , t
he other alibaba . cassim , the elder…"
vocab = {}
word_list = alibaba.split()
```

이제 분리된 단어와 빈도수를 `vocab` 딕셔너리에 넣어 줄 차례입니다. 현재 `word_list`에는 수많은 단어가 있습니다. `alibaba`를 예로 들어 보겠습니다. `alibaba`가 `vocab` 딕셔너리에 없다면 `alibaba`를 키로 추가해야 합니다. `alibaba`의 첫 등장이므로 빈도수는 당연히 1이 되죠. 이는 `vocab` 딕셔너리에 단어를 키로 하는 요소가 없다면 `vocab[단어] = 1`로 요소를 추가해야 한다는 뜻입니다. 두 번째 `alibaba`를 만나면 이미 딕셔너리에 `alibaba`를 키로 하는 요소가 있으므로 해당 요소의 빈도수에 1을 더해야 합니다. 즉, `vocab[단어] += 1`을 해야죠. 이 작업을 `word_list`에 들어 있는 모든 단어에서 실행합니다. 따라서 `word_list`의 첫 단어부터 마지막 단어까지 `vocab` 딕셔너리에 있는지 반복해서 확인하는 작업이 필요합니다. 이 부분을 정리하면 다음과 같습니다.

```
word_list의 첫 단어부터 마지막 단어까지 반복하기
    단어가 vocab 딕셔너리의 키에 없으면
        vocab[단어] = 1
    단어가 vocab 딕셔너리의 키에 있으면
        vocab[단어] += 1
```

정리한 내용을 코드로 작성하면 다음과 같습니다.

```
for w in word_list:
    if w not in vocab: # 단어가 vocab 딕셔너리의 키에 없으면
        vocab[w] = 1   # 새로운 요소 추가
    else:              # 단어가 vocab 딕셔너리의 키에 있으면
        vocab[w] += 1  # vocab[w]의 값 1 증가
print(vocab)
```

반복문에서 `word_list`에 담긴 단어를 `w` 변수에 차례대로 저장하며 반복문 내부에 있는 조건문을 실행합니다. 만약 `w`에 저장한 단어가 `vocab` 딕셔너리에 없으면 이 단어는 처음으로 등장하므로 새로 키를 추가하고 값에는 빈도수 1을 넣습니다. 만약 `w`에 저장한 단어가 이미 `vocab`에 있다면 빈도수만 1을 증가시킵니다. 반복이 끝나면 마지막에 `print(vocab)`을 실행해 `vocab` 딕셔너리에 담긴 내용을 확인합니다.

```
실행결과
{'in': 15, 'a': 24, 'town': 1, 'of': 33, 'persia': 1, 'lived': 2, 'two': 3, 'brothers': 1,
',': 103, 'sons': 1, 'poor': 3, 'man': 3…}
```

단어와 빈도수가 쌍으로 출력됩니다. 이제 빈도수를 기준으로 내림차순 정렬할 차례입니다. 이때 값만으로 정렬하는 것이 아니라 값을 기준으로 키-값 쌍으로 정렬한다는 점에 유의해야 합니다. 따라서 `sorted(vocab.items())`로 정렬해야 합니다. 그리고 키-값 쌍에서 값을 기준으로 정렬하므로 `key=operator.itemgetter(1)` 옵션을 줍니다. 또한 내림차순 정렬이므로 `reverse=True` 옵션도 추가해야 합니다.

```
import operator

print(sorted(vocab.items(), key=operator.itemgetter(1), reverse=True))
```

```
실행결과
[(',', 103), ('the', 78), ('.', 41), ('and', 37), ('to', 34), ('of', 33), ('a', 24), ('his',
22), ('he', 22), ('alibaba', 17)…]
```

실행결과를 보면 가장 많이 등장한 것은 단어가 아닌 콤마(,)고, 그다음으로 많이 등장한 단어는 `the`네요. 예상한 결과와 조금 다릅니다. 다음 단계에서 이 부분을 해결해 봅시다.

의미 없는 단어 제거하기

앞서 실행결과에서 확인했듯이 의미 없는 단어나 기호가 가장 많이 등장합니다. 문장 기호, 전치사, 관사 등은 단어장에 필요 없으니 이런 기호나 단어는 지워 주겠습니다. 필요 없는 단어를 하나의 리스트로 묶은 후, 이 단어들을 `vocab` 딕셔너리에서 삭제합니다.

> 제제쌤의 조언
> 자료실에 의미 없는 단어를 정리해 두었으니 그대로 복사해서 사용하면 됩니다.

```
meaningless = ['.', ',', '!', '?', 'the', 'he', 'and', 'to', 'a', 'of', 'was', 'in', 'had',
'for', 'it', 'that', 'but', 'as', 'with', 'at', 'i', 'into', 'be', 'this', 'me', 'from', 'the
n', 'him', 'his', 'her', 'she', 'they', 'them', 'you']
for word in meaningless:
    del vocab[word]
```

필요 없는 단어를 `meaningless` 리스트에 담습니다. 반복문을 활용해 `meaningless` 리스트의 단어를 차례대로 `word`에 넣은 후, `del`로 `vocab` 딕셔너리에서 삭제합니다.

필요 없는 단어를 모두 제거한 후 다시 값을 기준으로 키-값 쌍을 내림차순 정렬해 변수에 저장합니다.

```
import operator

vocab_final = sorted(vocab.items(), key=operator.itemgetter(1), reverse=True)
print(vocab_final)
```

```
실행결과
[('alibaba', 17), ('door', 8), ('oil', 8), ('one', 6), ('cassim', 6), ('took', 6), ('said',
6), ('out', 6), ('put', 6), ('open', 5), ('which', 5)…]
```

빈도수 없이 단어만 출력하고 싶으면 빈도수를 기준으로 정렬한 리스트에서 반복문을 사용해 키만 출력하면 됩니다.

```
for k, v in vocab_final:
    print(k, end=' ')
```

실행결과

```
alibaba door oil one cassim took said out put open which morgiana house captain went…
```

1분 퀴즈

2 앞에서 작성한 단어장 프로그램을 일부 수정해 단어만 알파벳순으로 정렬해 보세요.

실행결과

```
['accosting', 'admission', 'adopt', 'again', 'alibaba', 'all', 'allowed', 'am', 'among',
'any', 'appeared', 'approaching', 'armed', 'arose', 'assemble', 'assembled', 'asses', 'aw
ay', 'back', 'bade', 'bags', 'bed', 'before', 'best'… ]
```

정답 및 해설

7

마무리

1 딕셔너리

키(key)와 값(value)이 한 쌍으로 대응되는 데이터 타입

형식

```
딕셔너리명 = {키 : 값 , 키 : 값…}
```

2 데이터 변경, 추가, 삭제하기

구분	설명	사용법
값 변경하기	키가 딕셔너리에 이미 존재하면 대응되는 값이 변경된다.	`딕셔너리명[키] = 값`
키-값 추가하기	키가 딕셔너리에 없으면 새로운 키-값이 추가된다.	`딕셔너리명[키] = 값`
키-값 삭제하기	키와 해당 키에 대응되는 값을 삭제한다.	`del 딕셔너리명[키]`

3 딕셔너리 명령어

설명	사용법
딕셔너리의 키만 보여 주는 명령어	`딕셔너리명.keys()`
딕셔너리의 값만 보여 주는 명령어	`딕셔너리명.values()`
딕셔너리의 키-값 쌍을 보여 주는 명령어	`딕셔너리명.items()`

4 딕셔너리 정렬하기

정렬 대상(기준)	정렬 방법	사용법

키	오름차순	`sorted(딕셔너리명.keys())`
		또는
		`sorted(딕셔너리명)`

	내림차순	`sorted(딕셔너리명.keys(), reverse=True)`
		또는
		`sorted(딕셔너리명, reverse=True)`

값	오름차순	`sorted(딕셔너리명.values())`
	내림차순	`sorted(딕셔너리명.values(), reverse=True)`

키-값(키)	오름차순	`sorted(딕셔너리명.items())`
		또는
		`sorted(딕셔너리명.items(), key=operator.itemgetter(0))`

	내림차순	`sorted(딕셔너리명.items(), reverse=True)`
		또는
		`sorted(딕셔너리명.items(), key=operator.itemgetter(0), reverse=True)`

키-값(값)	오름차순	`sorted(딕셔너리명.items(), key=operator.itemgetter(1))`
	내림차순	`sorted(딕셔너리명.items(), key=operator.itemgetter(1), reverse=True`

Self Check

1 다음과 같이 요일을 키, 요일의 영어 약어를 값으로 하는 `day` 딕셔너리가 있습니다. 문제에 맞춰 코드를 작성해 보세요.

```
day = {'월요일': 'Mon', '화요일': 'Tue', '수요일': 'Wed', '목요일': 'Thu', '금요일': 'Fri'}
```

1) `day` 딕셔너리에 토요일-Sat, 일요일-Sun을 각각 키-값으로 하는 데이터를 추가하세요.
2) 일요일 키에 해당하는 값을 Sun에서 SUN으로 변경하세요.
3) `day` 딕셔너리에서 수요일-Wed 데이터를 삭제하세요.

2 다음과 같이 도서명과 도서 가격을 키-값으로 하는 `book` 딕셔너리가 있습니다. 제작비 상승으로 도서 가격을 인상해야 합니다. 도서 가격이 15,000원 이하면 10% 인상, 15,000원 초과면 5% 인상한다고 할 때, 인상된 가격으로 `book` 딕셔너리의 도서 가격을 변경하세요.

```
book = {'역사대모험': 20000,
        '영단어': 9000,
        '파이썬': 17000,
        '여행에세이': 22000,
        '삼국지': 33000
       }
```

`힌트` 조건문과 반복문을 활용해 보세요!

3 다음과 같이 오디션 프로그램의 참가자와 득표수를 키-값으로 하는 `ranking` 딕셔너리가 있습니다. `ranking` 딕셔너리를 득표수가 높은 참가자부터 낮은 참가자 순서대로 정렬하세요.

```
ranking = {'라이언': 956412,
           '어피치': 796354,
           '니니즈': 861832,
           '네오': 387896,
           '프로도': 534840
          }
```

정답 및 해설